Day 8 - OOP Fundamentals: Classes, self, Instances

Day 8 — OOP Fundamentals: Classes, `self`, Instances

Objectives

- Understand, from first principles, what a class actually is and why programmers use them
- Understand `self` well enough to never find it mysterious again
- Understand the difference between a class attribute (shared) and an instance attribute (private to one object), and the bug this distinction prevents
- Build a sense for when reaching for a class is the right call, versus when a plain dict or function is enough

Welcome to Object-Oriented Programming

Today starts **Object-Oriented Programming**, usually abbreviated **OOP** — a way of organizing code around "objects" that bundle together both data and the operations that work on that data, instead of keeping data and functions completely separate (which is roughly what you've been doing all of Week 1). You've actually been *using* objects this entire course without necessarily thinking of them that way — remember from Day 1, everything in Python is an object, including the lists and dicts you already know how to use, each with their own bundled-in methods like `.append()` and `.get()`. Today you learn how to design and build your *own* objects, tailored to whatever you're working on.

A class is a blueprint; an instance is a real thing built from that blueprint

Imagine an architect's blueprint for a house. The blueprint itself isn't a house you can live in — it's a *plan* describing what every house built from it will have (so many bedrooms, a kitchen in this spot, etc.). You can build many actual, physical houses from that one blueprint, and each one is independent — painting one house blue doesn't paint the others blue too.

A **class** is exactly this kind of blueprint, written in code. An **instance** is one actual object built from that blueprint — and you can create as many independent instances as you like from a single class.

```
class Dog:
    def __init__(self, name, breed):
        self.name = name
        self.breed = breed

    def bark(self):
        return f"{self.name} says Woof!"

rex = Dog("Rex", "Labrador")     # rex is an INSTANCE of the Dog class
fido = Dog("Fido", "Poodle")      # fido is a completely separate instance

print(rex.bark())     # Rex says Woof!
```

```
print(fido.bark())      # Fido says Woof!
```

`Dog` is the blueprint (the class); `rex` and `fido` are two separate, independent houses built from it (the instances). Both have a `.name`, a `.breed`, and a `.bark()` method available, because that's what the `Dog` blueprint specifies every `Dog` instance must have — but `rex.name` and `fido.name` hold different values, entirely independent of one another, exactly like two houses built from one blueprint can be painted different colors without affecting each other.

`self` — not magic, just an explicit reference to "this particular instance"

If you've glanced ahead at other languages before, you may have seen the word `this` used similarly — Python's `self` does the same job, but, true to Python's general philosophy (favor being explicit over implicit — you'll hear this idea again), it's not hidden or automatic in quite the same way; it's an ordinary parameter that you must write out yourself.

Here's the mechanism, precisely: when you write `rex.bark()`, Python actually does something equivalent to `Dog.bark(rex)` behind the scenes — it automatically passes `rex` in as the *first* argument to `bark`. That's what the parameter named `self` inside the method's definition actually receives: the specific instance the method was called on. This is exactly *why* every single method you write inside a class must list `self` as its very first parameter — it's not optional boilerplate; it's how the method gets access to the particular instance's own data (`self.name`, `self.breed`) at all.

```
rex.bark()          # Python internally treats this as: Dog.bark(rex)
Dog.bark(rex)         # ...which you could, in principle, write directly yourself -- it does the sa
```

You could technically name this first parameter anything you like — Python doesn't enforce the name `self` — but every single Python programmer, everywhere, names it `self` by convention, and deviating from that convention would make your code confusing to anyone else who reads it (including future-you). Always call it `self`.

`__init__` — setting up a brand new instance

`__init__` (pronounced "dunder init" — "dunder" is short for "double underscore," since it's surrounded by two underscores on each side; you'll meet more of these "dunder methods" tomorrow) is a special method that Python automatically runs the instant you create a new instance of a class. Its job is to set up that instance's starting data.

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

p = Point(3, 4)     # __init__ runs automatically here, with self=p (the new instance), x=3, y=4
print(p.x, p.y)        # 3 4
```

`self.x = x` is worth reading very carefully, since the same word `x` appears twice with two different meanings: the `x` on the right is the *parameter* (the value that was passed in when `Point(3, 4)` was called); `self.x` on the left is creating a brand-new piece of data that belongs specifically to *this instance*, named `x`, and setting it equal to that parameter's value. Any attribute you want each individual instance to have and remember

(accessible later as `some_instance.attribute_name`) needs to be set on `self` somewhere, almost always inside `__init__`.

Class attributes vs. instance attributes — a distinction that prevents a real, common bug

A **class attribute** is defined directly inside the class body, but *outside* of any method, and is shared by absolutely every instance of that class:

```
class Dog:
    species = "Canis familiaris"    # a CLASS attribute -- one single value, shared by ALL Dog insta

    def __init__(self, name):
        self.name = name              # an INSTANCE attribute -- each Dog gets its OWN separate valu
```

Every `Dog` instance reads the exact same `species` value (unless you deliberately override it on one specific instance), because there's only one `species` value total, living on the class itself, not on each instance individually. This is fine, even useful, for genuinely shared constants that will never differ between instances. But it becomes a serious, hard-to-spot bug the moment the class attribute is a *mutable* type (a list or dict) — and this is exactly the same underlying trap as Day 1's mutable-default-argument problem, now showing up in class form:

```
class ShoppingCart:
    items = []              # DANGER -- this ONE list is shared across EVERY ShoppingCart instance!

    def add(self, item):
        self.items.append(item)

cart1 = ShoppingCart()
cart2 = ShoppingCart()
cart1.add("apple")
print(cart2.items)   # ['apple']  !! -- cart2 never had "apple" added to it directly, yet there it
```

This happens because `items` was defined once, on the *class*, and both `cart1` and `cart2` share that exact same underlying list object — `cart1.add("apple")` mutates that one shared list in place, and `cart2` (which was never given its own separate list) sees the exact same shared object change. The fix mirrors Day 1's exactly: put mutable state inside `__init__`, so each instance gets its own, brand-new, independent object:

```
class ShoppingCart:
    def __init__(self):
        self.items = []       # a fresh, new, empty list -- created separately for EACH instance
```

Methods — functions that live inside a class and can read/change that instance's data

A **method** is simply a function defined inside a class. Because it always receives `self` (the specific instance it was called on) as its first parameter, a method can read and modify that particular instance's own data:

```
class Counter:
    def __init__(self):
        self.count = 0
```

```
        def increment(self):
            self.count += 1     # modifies THIS instance's own count -- other Counter instances are una

        def reset(self):
            self.count = 0

c1 = Counter()
c2 = Counter()
c1.increment()
c1.increment()
print(c1.count)   # 2
print(c2.count)    # 0 -- completely separate from c1, since each instance's __init__ gave it its o
```

Compare this with Day 4's closure-based counter (`make_counter`, which used `nonlocal`) — both achieve a similar goal (something that remembers a running value across calls), but a class scales far better once you have several *related* pieces of state and *several* different operations that need to work on that same state together — which is exactly what starts happening as your programs grow past today.

When should you actually reach for a class, versus a plain dict or a plain function?

This is a genuine judgment call you'll get better at with practice, but here's a starting rule of thumb:

- **Use a plain function** when you're just computing something from some inputs, with no ongoing state to remember between calls.
- **Use a plain dict** when you have a simple bundle of related data, but no real behavior (methods) attached to it, and you're not trying to enforce any particular guarantee about exactly which fields it will always have.
- **Use a class** once you have data *and* behavior that clearly belong together, and you want a guarantee about shape — every instance of `Dog` is guaranteed to have a `.name` and a `.bark()` method, in a way a loose dict never guarantees anything about its own keys. A strong, practical signal that you've outgrown a dict and should switch to a class: you notice you have several separate functions that all take the same dict shape as their first argument, and each one reaches in and reads or mutates specific keys of it — that's data and behavior that clearly belong bundled together, which is precisely what a class is for.

Exercises

Open `exercises.py`, fill in each `# TODO` inside the class definitions provided, and run `python exercises.py` to check your work against the PASS/FAIL output.